

Numerics II

Freie Universität Berlin

Winter semester 2025/2026

Dr. Robert Gruhlke, André-Alexander Zepernick

Exercise Sheet 5

Deadline: Friday, November 21st, 2025 at 10 am

Exercise 1 (3-step Adams-Bashforth method; 4 points)

Derive the 3-step Adams-Bashforth method from the formulas in the lecture notes.

Programming Exercise 1 (Implementation of embedded RK schemes; $4 + 4 = 8$ points)

In this exercise we extend the numerical integration of the van der Pol system, which was already investigated on exercise sheet 4, i.e.,

$$\begin{aligned}y_1'(t) &= y_2(t), \\y_2'(t) &= 10(1 - y_1(t)^2)y_2(t) - y_1(t), \quad t \in (0, 20], \\y_1(0) &= 0, \quad y_2(0) = 1,\end{aligned}$$

by implementing an *adaptive step-size control* based on an embedded Runge–Kutta scheme.

The adaptive controller follows the procedure:

Algorithm 1 Adaptive Step-Size Control

Require: current step size h , current value Y_k at time t_k , increment functions Φ and $\tilde{\Phi}$ of order q and $q+1$, and an error tolerance $\varepsilon > 0$.

Ensure: next value Y_{k+1} and next step size estimate h_{next} .

1: Compute the two trial solutions

$$Y = Y_k + h \Phi(t_k, Y_k, h), \quad \tilde{Y} = Y_k + h \tilde{\Phi}(t_k, Y_k, h).$$

2: Estimate the scaling factor

$$s = \left(\frac{h \varepsilon}{\|Y - \tilde{Y}\|_\infty} \right)^{1/q}.$$

3: **if** $s \geq 1$ **then**

4: Accept the step: $Y_{k+1} \leftarrow \tilde{Y}$, $t_{k+1} \leftarrow t_k + h$.

5: Update the step size estimate $h_{\text{next}} \leftarrow \min\{2, s\} h$.

6: **else**

7: Reject the step and repeat with $h \leftarrow \max\{1/2, s\} h$.

8: **end if**

a) Implement this algorithm in Python using the *embedded* RK4(3) scheme, which you derived on exercise sheet 4. Your function should have the interface

`solve_adaptive_rk43_procedure(f, y0, I, h0, epsilon, q),`

where $\mathbf{f}$ defines the right-hand side of the ODE, $\mathbf{y}_0$ is the initial value, and $I = [a, b]$ is the integration interval.

- b) Apply your implementation to the van der Pol system on $I = [0, 20]$ with parameters

$$h_0 = 10^{-4}, \quad \varepsilon = 10^{-4}.$$

- Plot the components $y_1(t)$ and $y_2(t)$ versus t .
- Compare the performance and accuracy of your adaptive scheme with a fixed-step classical RK4 method using step size $h = 0.0625$.
- Compute and plot a reference solution (e.g. classical RK4 with $h_{\text{ref}} = 2^{-14}$) and determine the maximal global error

$$\max_k \|Y_k - y_{\text{ref}}(t_k)\|_\infty$$

for both the adaptive and the fixed-step schemes. Report the total number of function evaluations and the observed errors.

Programming Exercise 2 (Adams-Bashforth method; $1 + 2 + 1 = 4$ points)

Consider the initial value problem from exercise sheet 2, i.e.,

$$y'(t) = -200t y(t)^2, \quad t \in [-1, 0], \quad y(-1) = \frac{1}{101},$$

whose exact solution is given by

$$y(t) = \frac{1}{100t^2 + 1}.$$

- a) Recall that the Adams-Bashforth method of order $m = 2$ for a constant step size h reads

$$Y_{k+1} = Y_k + h \left(\frac{3}{2}f(t_k, Y_k) - \frac{1}{2}f(t_{k-1}, Y_{k-1}) \right), \quad k = 1, \dots, N-1.$$

Since this is a two-step method, use the explicit Euler method to compute Y_1 from the initial condition:

$$Y_1 = Y_0 + h f(t_0, Y_0), \quad Y_0 = y(-1) = \frac{1}{101}.$$

- b) Implement this method in **Python** for the given problem on the interval $[-1, 0]$. Use a uniform grid with step size $h = \frac{1}{N}$ and grid points

$$t_k = -1 + kh, \quad 0 \leq k \leq N,$$

for $N \in \{25, 50, 100, 200, 400, 800, 1600\}$.

- c) For each N , compute the numerical solution $\{Y_k\}$ and the exact solution $\{y(t_k)\}$. Determine the maximum error

$$E_N = \max_{0 \leq k \leq N} |Y_k - y(t_k)|$$

and verify numerically that the Adams-Bashforth method of order two exhibits second-order convergence.

Rules for submission & passing criterion:

Exercises must be completed in **groups of three** and **submitted electronically via the Whiteboard system** under “**Assignments**.” Late submissions will not be corrected. Please **state the names of all group members** on the submission. The programming language of this course is **Python**. The exercises will be discussed in the tutorial the following week. To pass the tutorial component of the module, you must achieve **at least 50% of the homework points**.